

Perfilamento Paralelo de Grandes Datasets CSV Escalando o módulo DataSetSummary do framework VisFlow-MM

Otavio A. T. Fonseca

¹Universidade Federal de Lavras

otavio.fonseca1@estudante.ufla.br

Resumo. O VisFlow-MM é um framework de geração de visualizações a partir de linguagem natural (NL2Vis) que depende de um módulo determinístico de perfilamento de dados — a classe `DataSetSummary` — para extrair metadados estruturais e estatísticos de datasets CSV antes de qualquer geração de código. Este trabalho investiga como escalar esse módulo para grandes volumes de dados, adaptando-o a um pipeline de processamento em chunks paralelizado de duas formas: (1) `ProcessPoolExecutor` do Python e (2) Apache Spark em modo `local[N]`. Os experimentos utilizaram o dataset NYC Yellow Taxi (1 GB, 5 GB e 10 GB) em um AMD Ryzen 7 8700G com 16 núcleos lógicos. O multiprocessing Python não apresentou ganho com mais workers (speedup máximo: 0,93×), evidenciando gargalo de I/O. O Apache Spark demonstrou escalabilidade real: de 124,6 s com 1 núcleo a 23,3 s com 16 núcleos (speedup 5,35×), graças à ausência do GIL na JVM e à fusão de agregações em um único scan pelo Catalyst Optimizer.

Abstract. VisFlow-MM is a natural language-to-visualization (NL2Vis) framework that relies on a deterministic data profiling module — the `DataSetSummary` class — to extract structural and statistical metadata from CSV datasets prior to code generation. This work investigates how to scale this module to large data volumes, adapting it to a chunk-based parallel pipeline in two modes: (1) Python's `ProcessPoolExecutor` and (2) Apache Spark in `local[N]` mode. Experiments used the NYC Yellow Taxi dataset (1 GB, 5 GB, and 10 GB) on an AMD Ryzen 7 8700G with 16 logical cores. Python multiprocessing showed no performance gain with additional workers (maximum speedup: 0.93×), revealing an I/O-bound bottleneck. Apache Spark demonstrated real scalability: from 124.6 s with 1 core to 23.3 s with 16 cores (5.35× speedup), thanks to GIL-free JVM threads and the Catalyst Optimizer compiling all aggregations into a single data scan.

1. Introdução

A visualização de dados ocupa papel central na análise exploratória, transformando tabelas complexas em representações gráficas que revelam padrões, tendências e anomalias. Avanços recentes em Large Language Models (LLMs) impulsionaram o desenvolvimento de sistemas de geração automática de visualizações a partir de linguagem natural (NL2Vis), como Chat2Vis [Maddigan and Susnjak 2023], LIDA [Dibia 2023], NVAgent [Ouyang et al. 2025] e DeepVIS [Shuai et al. 2026]. Esses sistemas reduzem a barreira de entrada à análise de dados ao eliminar a necessidade de conhecimento de linguagens de programação ou gramáticas de visualização.

Nesse contexto, o **VisFlow-MM** [Fonseca 2026b] é um sistema híbrido de NL2Vis que integra pré-processamento determinístico, geração de código Python via LLM, validação por agentes e condicionamento multimodal por imagens de referência. Um componente crítico do VisFlow-MM é o módulo de perfilamento de dados: a classe `DataSetSummary`, que extrai metadados estruturais e estatísticos do dataset antes da geração de código. Esse sumário é enviado ao LLM como contexto, e sua qualidade impacta diretamente a precisão das visualizações geradas.

O problema é que o `DataSetSummary` original foi projetado para datasets que cabem inteiramente na RAM. Com o crescimento dos volumes de dados arquivos de dezenas de gigabytes são comuns em domínios como transporte urbano, saúde e financeiro, essa premissa torna-se inviável. Escalar o módulo de perfilamento sem perder precisão estatística, e sem romper a integração com o pipeline do VisFlow-MM, é o desafio endereçado por este trabalho.

Dois paradigmas de paralelismo são investigados neste trabalho. O primeiro é o *Multiprocessing* Python (`ProcessPoolExecutor`), no qual o CSV é lido em *chunks* de 500 000 linhas; cada *chunk* é enviado a um processo *worker* que computa estatísticas parciais, e um passo de agregação final combina os resultados. O segundo é o Apache Spark [Zaharia et al. 2016] (`local[N]`), no qual o mesmo perfilamento é reexpresso como transformações sobre um *DataFrame* Spark, onde o *Catalyst Optimizer* compila o plano de execução e *threads* Java executam em paralelo sem GIL.

As perguntas de pesquisa que orientam o estudo são: se adicionar *workers* Python reduz o tempo de perfilamento de CSVs grandes; se o Apache Spark apresenta melhor escalabilidade e por quê; e qual abordagem é mais adequada para integração com o VisFlow-MM em produção.

Os resultados revelam um comportamento contra-intuitivo: o *multiprocessing* Python não escala devido a gargalo de I/O, enquanto o Spark alcança *speedup* de $5,35\times$ com 16 núcleos, igualando em tempo o laço Python serial, mas com capacidade de continuar escalando em hardware com mais núcleos ou em *cluster*.

2. VisFlow-MM: Contexto e Papel do Pré-processamento

Em sistemas NL2Vis baseados em LLMs, o modelo nunca acessa os dados brutos diretamente: ele recebe uma representação intermediária construída por um módulo determinístico de pré-processamento. Essa mediação é necessária porque arquivos CSV reais excedem a janela de contexto dos modelos e carregam ruído (tipos heterogêneos, datas em formatos variados) que degrada a geração de código. Consequentemente, a qualidade e a escalabilidade do pré-processamento determinam o teto de desempenho de todo o pipeline, tornando esse componente um objeto de pesquisa de primeira ordem, e não mera infraestrutura.

2.1. Visão Geral do Framework

O VisFlow-MM [Fonseca 2026b] é um sistema NL2Vis híbrido que converte consultas em linguagem natural em programas Python executáveis capazes de gerar visualizações com a biblioteca Matplotlib. O framework é organizado em quatro estágios principais (Figura 1):

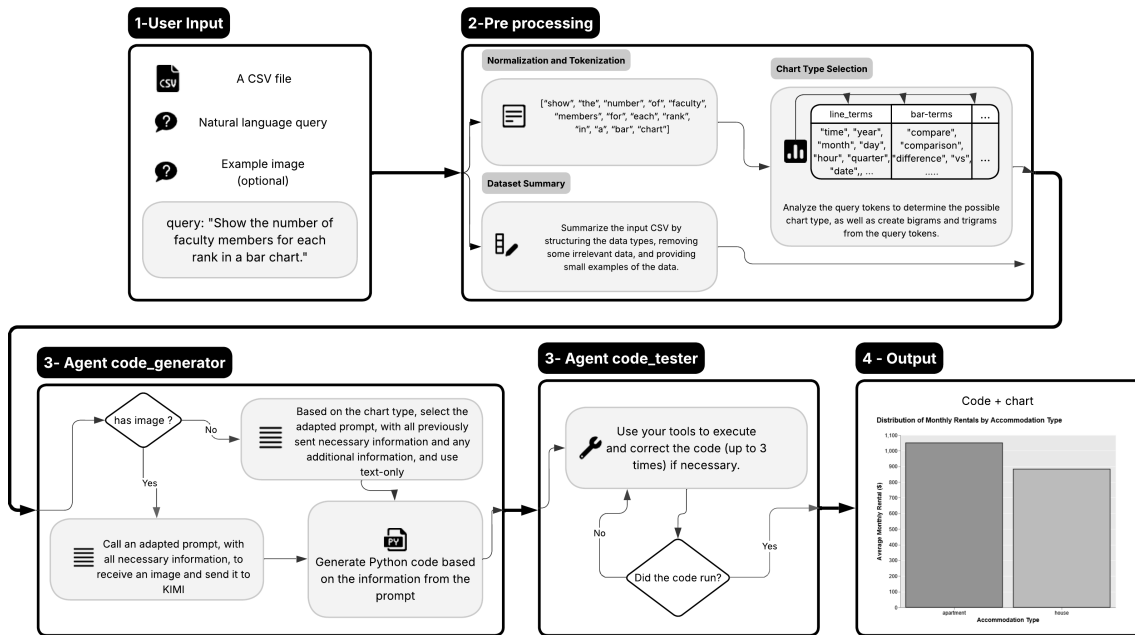


Figura 1. Visão geral do sistema proposto. O processo começa com a normalização da consulta em linguagem natural e a detecção do tipo de gráfico. Em seguida, o conjunto de dados é resumido e estruturado, preservando apenas as informações relevantes. O sistema então seleciona o modo de geração (apenas texto ou multimodal). Com base nessa decisão, um Modelo de Linguagem de Grande Escala (LLM) gera código Python executável, que é validado e salvo como um módulo reutilizável. Quando executado, esse código produz a visualização final como uma imagem PNG.

1. **Entrada do usuário:** consulta em linguagem natural, arquivo CSV e, opcionalmente, uma imagem de referência.
2. **Pré-processamento determinístico:** normalização da consulta, inferência do tipo de gráfico (*chart type selection*) e *sumarização do dataset* via `DataSetSummary`.
3. **Geração de código:** um LLM (modelo KIMI-K2.5) recebe o sumário do dataset, o tipo de gráfico, exemplos *few-shot* e, no modo multimodal, a imagem de referência; e gera um script Python.
4. **Validação e correção:** um agente executor testa o código gerado e aplica correções localizadas em até 3 tentativas.

2.2. O Módulo `DataSetSummary` no Pipeline

O `DataSetSummary` é o componente determinístico responsável por transformar o dataset CSV em uma representação compacta que serve de contexto para o LLM. Ele filtra colunas sensíveis, extrai metadados de esquema (tipos, cardinalidades, amostras) e produz estatísticas descritivas que orientam as decisões de agrupamento, eixos e agregações na visualização final.

A qualidade do sumário impacta diretamente a qualidade do código gerado: um sumário incompleto ou incorreto leva o LLM a gerar visualizações com eixos trocados, agregações erradas ou tipos de gráfico inadequados, os erros mais frequentes em sistemas NL2Vis [Wu et al. 2024]. Adicionalmente, estudos recentes mostram que LLMs têm difi-

culdade em calcular estatísticas precisas diretamente sobre dados brutos [Zhu et al. 2024], reforçando a necessidade de um módulo determinístico de perfilamento como substituto da inferência estatística pelo modelo.

2.3. Resultados do VisFlow-MM no Benchmark VisEval

A Tabela 1 resume os resultados do VisFlow-MM no benchmark VisEval [Chen et al. 2025], que avalia taxa de código inválido (*invalid rate*), taxa de saída ilegal (*illegal rate*) e taxa de aprovação (*pass rate*).

Tabela 1. Resultados no VisEval [Chen et al. 2025]: VisFlow-MM e sistemas NL2Vis recentes. Valores em negrito indicam o melhor por métrica.

Método	Inválido (%)	Ilegal (%)	Pass (%)
VisFlow-MM (text-only)	20,40	18,50	61,20
VisFlow-MM (multimodal)	0,13	19,69	80,18
LIDA [Dibia 2023]	1,13	21,20	77,66
Chat2Vis [Maddigan and Susnjak 2023]	0,86	21,37	77,75
NVAgent [Ouyang et al. 2025]	0,72	13,63	85,63

O condicionamento multimodal reduziu a taxa de código inválido de 20,40% para 0,13% e elevou a taxa de aprovação de 61,20% para 80,18%. O VisFlow-MM supera LIDA e Chat2Vis em taxa de aprovação e mantém-se competitivo com o NVAgent, que usa uma arquitetura multi-agente mais complexa.

2.4. Motivação para Escalar o DataSetSummary

O DataSetSummary original executa `pd.read_csv(path)` sem `chunksize`, carregando o dataset inteiro na RAM. Para datasets de 5–10 GB (comuns em domínios de transporte urbano, IoT e logs de sistema), esse comportamento esgota a memória disponível e inviabiliza o pipeline.

Escalar esse módulo, mantendo a exatidão estatística e a compatibilidade com a interface esperada pelo VisFlow-MM, é o problema central estudado neste artigo.

3. DataSetSummary: Lógica Original

A classe DataSetSummary executa sete etapas determinísticas sobre um DataFrame pandas, sintetizadas na Figura 2:

- Filtragem de colunas sensíveis:** expressão regular sobre nomes de colunas para remover campos que contenham `login`, `cpf` ou `cnnpj`, evitando vazamento de dados privados para o LLM.
- Dimensões e tipos:** número de linhas e colunas, tipos de dados por coluna (`df.dtypes`), contagem de nulos por coluna.
- Value counts categóricos:** `value_counts()` para colunas `object` com cardinalidade ≤ 50 , fornecendo ao LLM os valores mais frequentes para filtragens e agrupamentos.
- Deteção de colunas de data:** amostragem do primeiro valor não-nulo para identificar presença de separadores (`/`, `-`) e dígitos.

5. **Classificação de cardinalidade numérica:** colunas numéricas são classificadas em três categorias de acordo com o número de valores distintos, e o sumário é construído de forma diferente para cada uma:

- *Categórica discreta* (≤ 10 únicos): o sumário inclui o conjunto completo de valores e suas frequências absolutas. Sinaliza ao LLM que a coluna é adequada para agrupamento, filtragem ou eixo categórico, por exemplo, `payment_type` com 6 valores inteiros que representam formas de pagamento, onde um gráfico de barras é semanticamente correto [Zacks and Tversky 1999].
- *Semi-contínua* (11–100 únicos): inclui min, max, média e desvio-padrão, além de uma amostra dos valores mais frequentes. Indica variáveis ordinais ou discretas densas onde agrupamento por faixas (*bins*) é mais útil que enumeração individual, por exemplo, o número de passageiros (1–8).
- *Contínua* (> 100 únicos): inclui apenas estatísticas descritivas (min, max, média, σ). Orienta o LLM a usar histogramas, *scatter plots* ou eixos numéricos contínuos, por exemplo, `trip_distance` e `fare_amount`.

Sem essa classificação, o LLM tenderia a tratar variáveis como `payment_type` como contínuas, gerando histogramas onde gráficos de barras seriam semanticamente corretos.

6. **Estatísticas descritivas:** mínimo, máximo, média e desvio-padrão via `df.describe()`.

7. **Conjunto de valores únicos:** para colunas com ≤ 20 valores distintos, o conjunto completo é incluído no sumário.

O problema de escala surge porque a Etapa 1 do pipeline, `pd.read_csv(path)` sem `chunksize`, carrega o arquivo inteiro na RAM. Um CSV de 10 GB exige ao menos 10 GB de memória para o DataFrame, mais a memória das estruturas de profiling, tornando o processamento inviável em máquinas com 16–32 GB de RAM com outros processos em execução.

4. Arquitetura do Pipeline Paralelo

4.1. Processamento em Chunks

A solução central é substituir a leitura integral por leitura incremental:

Listing 1. Leitura incremental com chunksize

```
reader = pd.read_csv(path, chunksize=500_000, low_memory=False)
for chunk_df in reader:
    result = profile_chunk(chunk_df)
    partial_results.append(result)
```

Cada chunk contém 500 000 linhas (≈ 90 MB de RAM), independente do tamanho total do arquivo. As estatísticas de cada chunk são acumuladas e combinadas em um passo de agregação final.

4.2. Agregação Exata entre Chunks

Um dos desafios centrais do processamento em chunks é combinar os resultados parciais em um sumário global *numericamente idêntico* ao que seria obtido com o dataset inteiro na memória. A dificuldade surge porque estatísticas como média e desvio-padrão **não são lineares na concatenação**: não basta calcular a média das médias parciais.

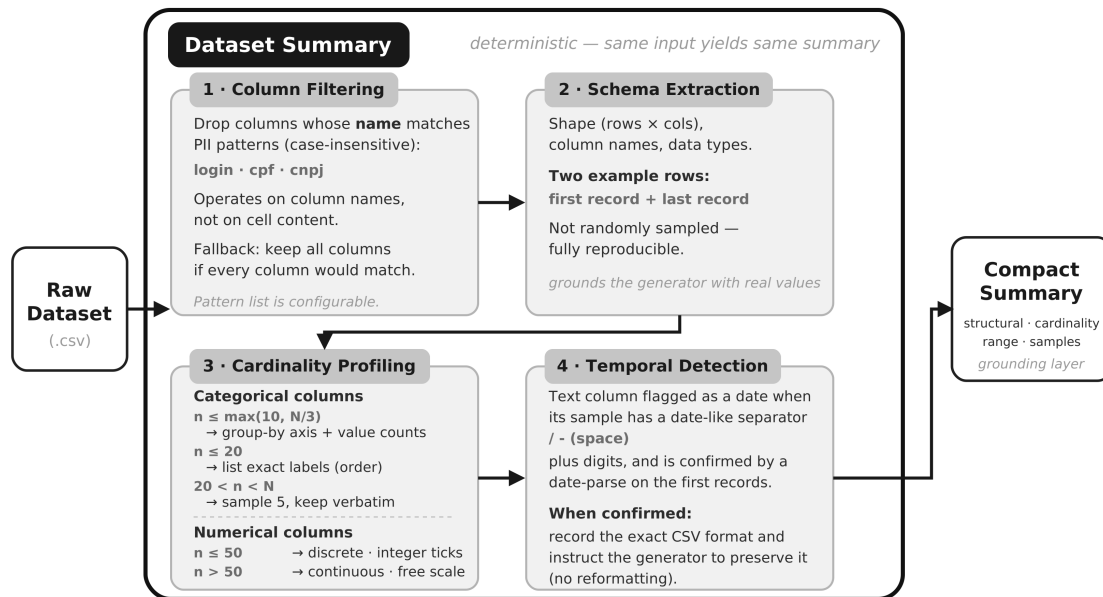


Figura 2. Construção do Dataset Summary. O CSV bruto percorre quatro etapas determinísticas: (1) filtragem de colunas, que descarta atributos cujo *nome* casa com padrões de PII; (2) extração de esquema (formato, nomes e tipos das colunas, mais dois registros de exemplo); (3) perfilamento de cardinalidade, que separa colunas categóricas e numéricas e fixa a estratégia de codificação; e (4) detecção temporal, que identifica e preserva o formato de datas. O resultado é um resumo compacto que ancora o gerador com valores reais, garantindo reprodutibilidade: a mesma entrada gera sempre o mesmo resumo.

Por que a média das médias está errada. Considere dois chunks: o primeiro tem $n_1 = 3$ registros com valores $\{10, 10, 10\}$ (média parcial = 10) e o segundo tem $n_2 = 1$ registro com valor $\{20\}$ (média parcial = 20). A média global *correta* pondera pelo tamanho de cada chunk: $(3 \times 10 + 1 \times 20) / (3 + 1) = 12,5$. A média das médias parciais daria $(10 + 20) / 2 = 15$, erro de 20%. O problema é análogo para o desvio-padrão, com o agravante de ser mais difícil de perceber visualmente.

Estatísticas suficientes por chunk. Para cada chunk k com n_k linhas, o worker armazena dois escalares por coluna numérica, suficientes para reconstruir média e desvio-padrão globais sem guardar nenhum valor individual:

- $S_k = \sum_{i \in C_k} x_i$ — a **soma** de todos os valores do chunk. Ao final, a soma de todas as S_k dividida pelo total de registros $\sum_k n_k$ dá a média global ponderada pelo tamanho de cada chunk.
- $Q_k = \sum_{i \in C_k} x_i^2$ — a **soma dos quadrados** dos valores. Combinada com a média, permite calcular o desvio-padrão pela identidade de König–Huygens ($\text{Var}(X) = E[X^2] - (E[X])^2$), sem precisar de uma segunda passagem pelo dado.

Esses dois valores são chamados de *estatísticas suficientes*: contêm toda a informação necessária para reconstruir os parâmetros globais, independente de quantos chunks existam.

Fórmulas de agregação global. A média global é a média ponderada dos somatórios:

$$\bar{x} = \frac{\sum_k S_k}{\sum_k n_k} \quad (1)$$

O desvio-padrão usa a identidade $\sigma^2 = E[X^2] - \bar{x}^2$, onde $E[X^2]$ é a média global de x^2 , obtida exatamente da mesma forma:

$$\sigma = \sqrt{\frac{\sum_k Q_k}{\sum_k n_k} - \bar{x}^2} \quad (2)$$

Não há arredondamento acumulado nem segunda passagem pelo dataset: cada chunk contribui com dois escalares (S_k, Q_k) e os resultados são combinados em $O(K)$, onde K é o número de chunks.

Agregação das demais métricas. Cada métrica do `DataSetSummary` tem sua própria regra de combinação:

- **Mínimo e máximo globais:** $\min_{\text{global}} = \min_k(\min_k)$ e idem para o máximo — trivialmente exatos.
- **Value counts** (frequências categóricas): adição de objetos `Counter` do Python, equivalente a somar as frequências de cada valor em todos os chunks — o resultado é idêntico ao `value_counts()` sobre o dataset inteiro.
- **Conjuntos de valores únicos:** união de `sets` Python preserva exatamente os valores distintos, sem duplicatas.
- **Nulos:** soma da contagem de nulos por coluna em cada chunk.
- **Formato de data:** preservado do primeiro chunk não-nulo (o formato é consistente dentro de um mesmo dataset).

Essa abordagem garante que o sumário paralelo seja **estatisticamente idêntico** ao do `DataSetSummary` original, mantendo compatibilidade total com o pipeline do VisFlow-MM [Fonseca 2026b].

4.3. Janela Deslizante de Memória

Para evitar estouro de RAM com datasets de 10 GB e múltiplos workers em paralelo, implementa-se uma janela deslizante que mantém no máximo $2 \times N$ chunks pendentes simultaneamente:

Listing 2. Sliding window com futures

```
MAX_PENDING = n_workers * 2
futures = {}
for chunk_id, chunk in enumerate(reader):
    if len(futures) >= MAX_PENDING:
        done, _ = wait(futures, return_when=FIRST_COMPLETED)
```

```

for f in done:
    results.append(f.result())
    del futures[f]
futures[executor.submit(profile_chunk, (chunk_id, chunk))] =
    chunk_id

```

4.4. Implementação com Apache Spark

A implementação Spark replica as 7 etapas do DataSetSummary usando a API PySpark DataFrame. A diferença fundamental está em *quem* faz a leitura e *como* as agregações são executadas (Tabela 2).

Tabela 2. Diferenças técnicas entre Multiprocessing Python e Apache Spark

Aspecto	Multiprocessing	Apache Spark
Leitura do CSV	1 thread serial (pandas)	Threads JVM paralelas
Paralelismo	Processos Python	Threads Java (sem GIL)
Serialização	pickle (≈ 90 MB/chunk)	Memória JVM off-heap
Agregações	Passes múltiplos por chunk	1 job Catalyst (scan único)
Cardinalidade	nunique() exato	HyperLogLog ($\pm 5\%$)
Overhead inicial	$\approx 0,1$ s	10–15 s (JVM) excluído

No Spark, todas as agregações numéricas são compiladas pelo Catalyst Optimizer em um único job que percorre o dataset uma única vez:

Listing 3. Agregações Spark em scan único

```

num_exprs = []
for col in numeric_cols:
    num_exprs += [
        F.min(col).alias(f"min__{col}"),
        F.max(col).alias(f"max__{col}"),
        F.mean(col).alias(f"mean__{col}"),
        F.stddev(col).alias(f"std__{col}"),
        F.approx_count_distinct(col, rsd=0.05).alias(f"uniq__{col}"),
    ]
df.agg(*num_exprs).collect()    # 1 unico job

```

5. Configuração Experimental

5.1. Dataset: NYC Yellow Taxi

Os dados de corridas de táxi amarelo de Nova York são disponibilizados pela NYC TLC [NYC Taxi & Limousine Commission 2024] em formato Parquet (2022–2024). Foram convertidos para CSV e concatenados em três tamanhos:

- **1 GB** (≈ 1069 MB): 3 meses, ≈ 8 milhões de linhas.
- **5 GB** (≈ 4585 MB): 14 meses, ≈ 49 milhões de linhas.

- **10 GB** (≈ 8910 MB): 27 meses, ≈ 94 milhões de linhas.

O dataset possui 19 colunas com tipos heterogêneos: numéricas (distância, tarifas, gorjetas), categóricas (método de pagamento, tipo de táxi) e temporais (horário de embarque e desembarque). Essa heterogeneidade exercita todas as 7 etapas do `DataSetSummary`.

5.2. Ambiente e Protocolo

Hardware: AMD Ryzen 7 8700G (8 cores / 16 threads), 64 GB DDR5, NVMe SSD.

Software: Python 3.11, pandas 2.2.2, PySpark 3.5.1 + Java 17, Docker Compose (reprodutibilidade). O código-fonte e os scripts de reprodução dos experimentos estão disponíveis em [Fonseca 2026a].

Protocolo: Para multiprocessing, foram testados $N \in \{1, 2, 4, 8, 16\}$ workers, com $N = 1$ executando em loop serial puro (baseline), 3 repetições cada. Para Spark, os mesmos níveis de paralelismo foram testados com `local [N]`, precedidos de 1 execução de *warmup* (JVM não contabilizada), 3 repetições. O tempo medido inclui leitura do CSV + todas as agregações + coleta dos resultados.

6. Resultados

6.1. Multiprocessing Python

Tabela 3. Multiprocessing Python: tempo médio (s) e speedup por dataset

Workers	1 GB		5 GB		10 GB	
	Tempo (s)	Speedup	Tempo (s)	Speedup	Tempo (s)	Speedup
1	23,48	1,00	101,20	1,00	192,27	1,00
2	24,65	0,95	96,14	1,05	180,81	1,06
4	25,17	0,93	96,51	1,05	182,25	1,06
8	25,47	0,92	96,81	1,04	181,53	1,06
16	25,40	0,92	103,31	0,98	181,65	1,06

Para o dataset de 1 GB, adicionar workers *piorou* o desempenho ($\text{speedup} < 1$): o overhead de criação de processos e serialização via pickle supera qualquer ganho de paralelismo. Para 5 GB e 10 GB, há um speedup marginal ($\approx 1,05$ – $1,06\times$) com 2–8 workers, que não cresce com mais workers. Em todos os cenários, a vazão de disco permanece constante em ≈ 45 – 47 MB/s, a velocidade máxima de leitura sequencial do SSD.

6.2. Apache Spark

O Spark escala de forma real e progressiva: speedup de $1,90\times$ com 2 núcleos a $5,35\times$ com 16 núcleos. Com 16 núcleos, o Spark (23,3 s) atingiu exatamente o tempo do Python serial (23,5 s), o **ponto de cruzamento**. A eficiência decresce com mais núcleos ($100\% \rightarrow 33\%$), confirmando a existência de uma fração serial irreduzível prevista pela Lei de Amdahl [Amdahl 1967].

Tabela 4. Apache Spark local [N] — dataset de 1 GB

Núcleos	Tempo (s)	Speedup	Eficiência (%)	Vazão (MB/s)
1	124,55	1,00	100,0	8,6
2	65,67	1,90	94,9	16,3
4	39,76	3,13	78,3	26,9
8	30,13	4,13	51,7	35,5
16	23,28	5,35	33,4	45,9

6.3. Visualização Comparativa

As Figuras 3 e 4 sintetizam graficamente o contraste entre as duas abordagens. A Figura 3 apresenta as curvas de speedup em função do número de workers/núcleos, contrastando-as com o speedup ideal (linear) e com a previsão da Lei de Amdahl. A Figura 4 mostra a vazão efetiva de leitura, evidenciando a saturação do multiprocessing no teto do SSD frente ao crescimento progressivo do Spark.

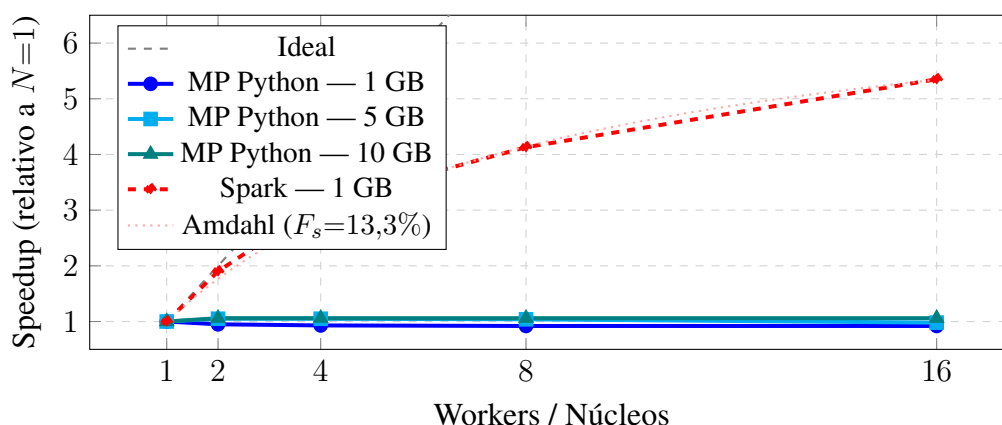


Figura 3. Curvas de speedup. O multiprocessing Python permanece plano independente do tamanho do dataset. O Spark cresce até 5,35× com 16 núcleos.

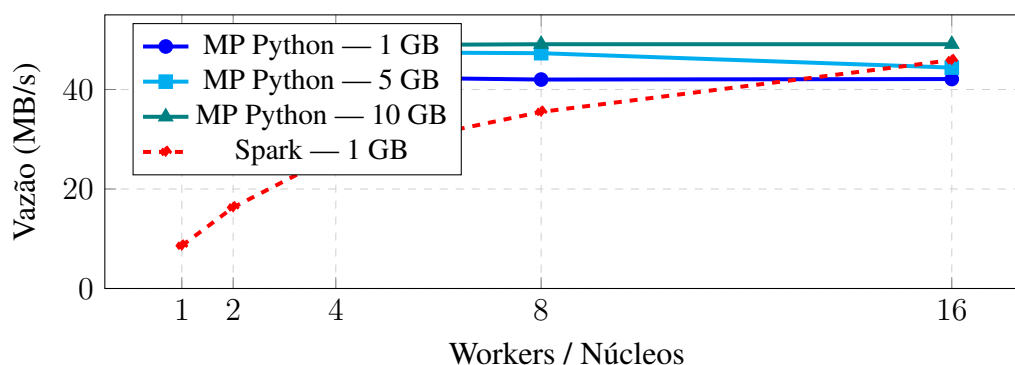


Figura 4. Vazão efetiva. O multiprocessing satura no teto do SSD (≈ 47 MB/s). O Spark cresce de 8,6 a 45,9 MB/s ao usar mais núcleos.

7. Análise e Discussão

7.1. Por Que o Multiprocessing Não Escala

O processo principal Python lê o CSV sequencialmente a ≈ 47 MB/s. As operações de perfilamento por chunk completam em décimos de segundo, muito mais rápido do que o disco entrega o próximo chunk. Workers ficam ociosos aguardando dados: é um workload *I/O-bound*.

Além disso, o `ProcessPoolExecutor` serializa cada chunk via `pickle` (≈ 90 MB por chunk) para transferi-lo ao processo worker via IPC, consumindo tempo que anula qualquer ganho.

A Lei de Amdahl [Amdahl 1967] formaliza o limite:

$$S(n) = \frac{1}{F_s + (1 - F_s)/n} \quad (3)$$

Com fração serial $F_s \approx 90\%$ (I/O), o speedup máximo teórico com $n \rightarrow \infty$ é $S(\infty) = 1/0,90 \approx 1,11\times$ compatível com os dados observados ($\leq 1,06\times$).

7.2. Por Que o Spark Escala

Threads Java sem GIL. O CPython bloqueia a execução paralela de threads via GIL. Processos Python separados (multiprocessing) evitam o GIL, mas mantêm a leitura serial no processo principal. Threads Java executam I/O e CPU em paralelo real, sem restrição equivalente.

Catalyst Optimizer (scan único). Enquanto o pandas computa cada agregação em passes separados por chunk, o Catalyst funde `min`, `max`, `mean`, `stddev` e `approx_count_distinct` em um único job que percorre o dado *uma vez*.

Ajustando a Lei de Amdahl aos dados do Spark ($S(16) = 5,35$, $n = 16$): $F_s \approx 13,3\%$, com speedup máximo teórico de $1/0,133 \approx 7,5\times$. Comparado ao multiprocessing ($F_s \approx 90\%$, teto $\approx 1,1\times$), o Spark reduz dramaticamente a fração serial ao paralelizar o I/O.

7.3. Custo de Entrada e Ponto de Cruzamento

Spark com 1 núcleo é $5,3\times$ mais lento que Python serial (124,6 s vs 23,5 s): a JVM tem overhead de planejamento (Catalyst), alocação de memória off-heap (Tungsten) e agendamento de tasks. Com 16 núcleos, esse overhead é amortizado e o Spark iguala o Python serial. Em hardware com 32+ núcleos ou em cluster distribuído, o Spark seria mais rápido.

7.4. Implicações para o VisFlow-MM

Para integrar o pré-processamento escalável ao VisFlow-MM, a recomendação prática depende do cenário de implantação:

Tabela 5. Guia de escolha: multiprocessing vs Spark para o DataSetSummary

Cenário	Recomendação	Motivo
Dataset < 5 GB, máquina simples	Python chunks serial	Menor overhead
Dataset 5–50 GB, 16+ cores	Spark <code>local [N]</code>	Escala com núcleos
Dataset > 50 GB ou cluster	Spark distribuído	I/O paralelo entre nós

8. Ameaças à Validade

Validade interna. Experimentos em máquina única com processos em segundo plano podem introduzir variabilidade. O outlier observado em $N = 16$ workers no dataset de 5 GB (113 s vs ≈ 97 s nas outras repetições) foi mantido por honestidade.

Validade externa. Resultados são específicos ao hardware testado (SSD NVMe, ≈ 47 MB/s, 16 cores). Em HDDs, o gargalo de I/O seria mais dominante; em SSDs mais rápidos, o crossover entre Spark e Python ocorreria com mais núcleos.

Generalidade. O workload de perfilamento é predominantemente de leitura com agregações simples (sem joins ou shuffles pesados). Workloads com shuffles extensos favoreceriam ainda mais o Spark.

9. Conclusão

Este trabalho escalonou o módulo `DataSetSummary` do framework VisFlow-MM [Fonseca 2026b] para processar grandes datasets CSV (até 10 GB) sem carregar o arquivo inteiro na RAM, comparando duas abordagens de paralelismo.

O multiprocessing Python não escalou: a leitura sequencial do CSV pelo processo principal limita o speedup a $\leq 1,06\times$, independente do número de workers. O Apache Spark demonstrou speedup real de $5,35\times$ com 16 núcleos no dataset de 1 GB, reduzindo a fração serial de $\approx 90\%$ (Python) para $\approx 13\%$ (Spark). Com 16 núcleos, ambos atingem ≈ 23 s, mas apenas o Spark continua escalando com mais recursos.

A lição central: **paralelizar computação sem paralelizar I/O é insuficiente para workloads de leitura de arquivos locais.** Para o VisFlow-MM processar datasets de produção de dezenas de gigabytes, o Spark em modo local (máquina com 16+ cores) ou em cluster distribuído é a abordagem recomendada, com a ressalva do overhead de inicialização da JVM que penaliza datasets pequenos.

Como trabalhos futuros: (1) executar o benchmark Spark para 5 GB e 10 GB; (2) avaliar o impacto do `chunksize` no multiprocessing; (3) integrar o pipeline paralelo como módulo opcional do VisFlow-MM, ativado automaticamente quando o dataset exceder um limiar configurável.

Referências

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proc. AFIPS Spring Joint Computer Conference*, pages 483–485.

- Chen, N., Zhang, Y., Xu, J., Ren, K., and Yang, Y. (2025). VisEval: A benchmark for data visualization in the era of large language models. *IEEE Trans. Vis. Comput. Graph.*, 31(1):1301–1311.
- Dibia, V. (2023). LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In *Proc. 61st Annual Meeting of the ACL (System Demonstrations)*, pages 113–126.
- Fonseca, O. (2026a). Perfilamento paralelo com multiprocessing e Apache Spark: código-fonte dos experimentos. <https://github.com/ootaviofonseca/pdm>. PCC557/PDM/UFLA, 2026. Acesso em: jun. 2026.
- Fonseca, O. (2026b). VisFlow-MM: Agentic multimodal chart generation with vision-language models. <https://github.com/ootaviofonseca/VISFLOW-MM>. Trabalho submetido, PPGCC/UFLA, 2026.
- Maddigan, P. and Susnjak, T. (2023). Chat2VIS: Generating data visualizations via natural language using ChatGPT, Codex and GPT-3 large language models. *IEEE Access*, 11:45181–45193.
- NYC Taxi & Limousine Commission (2024). TLC trip record data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Acesso em: jun. 2024.
- Ouyang, G., Chen, J., Nie, Z., Gui, Y., Wan, Y., Zhang, H., and Chen, D. (2025). NVAgent: Automated data visualization from natural language via collaborative agent workflow. In *Proc. 63rd Annual Meeting of the ACL*, pages 19534–19567, Vienna, Austria.
- Shuai, Z., Li, B., Yan, S., Luo, Y., and Yang, W. (2026). DeepVIS: Bridging natural language and data visualization through step-wise reasoning. *IEEE Trans. Vis. Comput. Graph.*, 32(1):868–878.
- Wu, Y., Wan, Y., Zhang, H., Sui, Y., Wei, W., Zhao, W., Xu, G., and Jin, H. (2024). Automated data visualization from natural language via large language models: An exploratory study. *Proc. ACM Manag. Data*, 2(3):115:1–115:28.
- Zacks, J. and Tversky, B. (1999). Bars and lines: A study of graphic communication. *Memory & Cognition*, 27(6):1073–1079.
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., and Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.
- Zhu, Y., Du, S., Li, B., Luo, Y., and Tang, N. (2024). Are large language models good statisticians? In *Proc. 38th Conference on Neural Information Processing Systems (NeurIPS)*.